

```

"""
Semi-Cascaded SLU on SLURP -- performance-
optimized version
=====

This is a drop-in replacement for the
original training script with the
single-CPU-core bottleneck removed. Summary
of what changed and why (see
inline comments tagged "# PERF:" for exact
locations):

1. ASR transcription
(`ASRTranscriber.build_cache`) now
decodes/resamples
    audio in a multi-process pool instead
of a serial list comprehension,
    so CPU-side audio I/O overlaps with
GPU-side Whisper decoding instead
of blocking it.

2. The frozen Whisper acoustic encoder is
only ever run ONCE per
    recording now
(`build_acoustic_feature_cache`), not once
per training
    step per epoch. Since
`freeze_whisper=True` means its output for
a

```

given waveform never changes, recomputing it every forward pass (the original behavior) wasted a full Whisper-encoder forward pass on every single step of every epoch. Features are cached to disk as fp16 tensors and just loaded (not recomputed) at train time.

3. The word-level Levenshtein tag alignment (``_align_and_project_tags``) is a deterministic, pure-Python, CPU-bound function of (`gt_words`, `gt_tags`, `asr_text`). The original code re-ran it inside ``collate_fn`` on every batch of every epoch. It's now computed once per recording, in parallel across CPU cores, and cached to disk.

4. DataLoaders use ``persistent_workers=True`` + ``pin_memory=True`` so worker processes aren't torn down/rebuilt every epoch and host->GPU copies are faster. Worker cost is now trivial (just loading a small cached tensor + tokenizing text) since (1)-(3) moved the expensive work out of the training loop entirely.

5. ``WhisperForConditionalGeneration.generate``.

```
..) ` now explicitly pins
    `num_beams=1` (greedy) for the ASR
pass. Some Whisper checkpoints'
    `generation_config` defaults to beam
search, which multiplies ASR
    decode time by the beam width for no
accuracy benefit in this
    use case (we only need a plausible
noisy hypothesis, not the best
    possible transcript).
```

Net effect: audio decode + Whisper-encoder
+ alignment work that used to
happen $O(\text{epochs})$ times now happens $O(1)$
times, in parallel, before
training starts. Training itself becomes
GPU-bound instead of
single-core-CPU-bound.

Everything else (model architecture,
losses, metrics) is unchanged from
the original script.

"""

```
import hashlib
import json
import os
import random
import re
from concurrent.futures import
ProcessPoolExecutor
from dataclasses import dataclass, field
from typing import Dict, List, Optional,
Tuple
```

```

# Must run before any
`transformers`/`huggingface_hub` import.
from hf_offline_utils import
enable_offline_mode
enable_offline_mode()

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchaudio
from torch.utils.data import Dataset,
DataLoader
from transformers import (
    WhisperModel,
    WhisperFeatureExtractor,
    WhisperForConditionalGeneration,
    WhisperProcessor,
    AutoModel,
    AutoTokenizer,
)

#
=====
===== #
# 0. Config
#
=====
===== #
@dataclass
class SLURPConfig:
    root_dir: str = "slurp_dataset"
    train_jsonl: str =

```

```

"dataset/slurp/train.jsonl"
    valid_jsonl: str =
"dataset/slurp/devel.jsonl"
    test_jsonl: str =
"dataset/slurp/test.jsonl"
    train_synthetic_jsonl: str =
"dataset/slurp/train_synthetic.jsonl"
    include_synthetic: bool = False

    audio_real_subdir: str = "slurp_real"
    audio_synth_subdir: str = "slurp_synth"

    recording_filter: str =
"correct_only"    # "correct_only" | "all"
    recording_type_filter: Optional[str] =
None

    sample_rate: int = 16000
    max_audio_seconds: float = 11.0

    whisper_model_name: str =
"openai/whisper-small"
    bert_model_name: str = "bert-base-
uncased"
    whisper_layer: int = 6
    freeze_whisper: bool =
True                # PERF: now a real
config, feature caching only valid when
True

    # Which cross-attention architecture to
use. See CrossAttentionBlock /
SemiCascadedSLU_SLURP:
    #    "text_query"    -- baseline

```

```

(original script): text tokens attend into
acoustic frames.
#                               Fused sequence is
word-level; feeds both the slot head and
(pooled) intent head.
#   "audio_query" -- full swap:
acoustic frames attend into text. Fused
sequence is frame-level;
#                               feeds the
(pooled) intent head directly. A secondary
text-query block then
#                               attends into that
frame-level representation to recover word-
level features
#                               for the slot head
(frame-level output alone has no notion of
"one tag per word").
#   "dual_branch" -- both blocks in
parallel: text-query branch feeds the slot
head (same as
#                               "text_query"),
audio-query branch feeds the intent head
(same pooling as
#                               "audio_query").
No secondary projection needed since each
branch only
#                               serves the head
suited to its native granularity.
fusion_mode: str = "text_query"
comparison_modes: Tuple[str, ...] =
("text_query", "audio_query",
"dual_branch")

asr_cache_path: str =

```

```

"slurp_asr_cache.json"
    use_ground_truth_transcript: bool =
False

    # PERF: caching knobs
    use_precomputed_acoustic_features: bool
= True
    acoustic_feature_cache_dir: str =
"slurp_acoustic_feature_cache"
    slot_tag_cache_path: str =
"slurp_slot_tag_cache.json"
    prep_num_workers: int =
max(os.cpu_count() or 4, 1)
    prep_batch_size: int =
32                # batch size used only
during the one-time caching passes

    max_text_len: int = 64
    batch_size: int = 8
    num_workers: int =
4                # DataLoader workers
during actual training
    lr: float = 3e-5
    epochs: int = 10
    slot_loss_weight: float = 1.0

    device: str = "cuda" if
torch.cuda.is_available() else "cpu"

#
=====
===== #
# 1. jsonl reading + sentence_annotation

```

```

parsing
#
=====
===== #
def _read_jsonl(path: str) -> List[dict]:
    entries = []
    with open(path) as f:
        for line in f:
            line = line.strip()
            if line:
                entries.append(json.loads(line))
    return entries

_ENTITY_PATTERN = re.compile(r"\s*([a-zA-Z_]+\s*:\s*([^\s]+?)\s*\s)")

def
parse_sentence_annotation(sentence_annotation: str) -> Tuple[List[str], List[str]]:
    words, tags = [], []
    pos = 0
    for m in
_ENTITY_PATTERN.finditer(sentence_annotation):
        before =
sentence_annotation[pos:m.start()]
        for w in before.split():
            words.append(w)
            tags.append("0")

        ent_type = m.group(1).strip()

```



```

        filler_words =
m.group(2).strip().split()
        for i, w in
enumerate(filler_words):
            tags.append(f"B-{{ent_type}}" if
i == 0 else f"I-{{ent_type}}")
            words.append(w)

        pos = m.end()

        after = sentence_annotation[pos:]
        for w in after.split():
            words.append(w)
            tags.append("0")

    return words, tags

#
=====
===== #
# 2. Word-level Levenshtein alignment
#
=====
===== #
def _align_and_project_tags(gt_words:
List[str], gt_tags: List[str], hyp_words:
List[str]) -> List[str]:
    n, m = len(gt_words), len(hyp_words)
    if m == 0:
        return []

    dp = [[0] * (m + 1) for _ in range(n +
1)]

```

```

    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            cost = 0 if gt_words[i -
1].lower() == hyp_words[j - 1].lower() else
1
            dp[i][j] = min(dp[i - 1][j] +
1, dp[i][j - 1] + 1, dp[i - 1][j - 1] +
cost)

    i, j = n, m
    hyp_tags = ["0"] * m
    while i > 0 or j > 0:
        if i > 0 and j > 0 and dp[i][j] ==
dp[i - 1][j - 1] + (
            0 if gt_words[i - 1].lower() ==
hyp_words[j - 1].lower() else 1
        ):
            hyp_tags[j - 1] = gt_tags[i -
1]
            i -= 1
            j -= 1
        elif i > 0 and dp[i][j] == dp[i -
1][j] + 1:
            i -= 1
        else:
            j -= 1
    return hyp_tags

# PERF: top-level, picklable worker for

```

```

ProcessPoolExecutor -- computes one
# example's alignment. Kept separate from
# `_align_and_project_tags` so it
# can be mapped in parallel over the whole
# dataset once, instead of being
# called serially inside collate_fn on
# every batch of every epoch.
def _align_worker(args: Tuple[List[str],
List[str], List[str]]) -> List[str]:
    gt_words, gt_tags, hyp_words = args
    return
_align_and_project_tags(gt_words, gt_tags,
hyp_words)

#
=====
===== #
# 3. Label vocab
#
=====
===== #
class SLURPLabelVocab:
    def __init__(self, jsonl_paths:
List[str]):
        intents = set()
        slot_types = set()
        for p in jsonl_paths:
            for entry in _read_jsonl(p):

intents.add(entry["intent"])
            _, tags =
parse_sentence_annotation(entry.get("senten
ce_annotation", entry["sentence"]))

```

```

        for t in tags:
            if t != "0":

slot_types.add(t[2:])

        self.intent2id = {v: i for i, v in
enumerate(sorted(intents))}
        self.id2intent = {i: v for v, i in
self.intent2id.items()}

        slot_labels = ["0"] + sorted(f"
{prefix}-{t}" for t in sorted(slot_types)
for prefix in ("B", "I"))
        self.slot2id = {v: i for i, v in
enumerate(slot_labels)}
        self.id2slot = {i: v for v, i in
self.slot2id.items()}

    @property
    def num_intents(self) -> int:
        return len(self.intent2id)

    @property
    def num_slot_labels(self) -> int:
        return len(self.slot2id)

    def save(self, path: str):
        with open(path, "w") as f:
            json.dump({"intent2id":
self.intent2id, "slot2id": self.slot2id},
f, indent=2)

    @classmethod
    def load(cls, path: str) ->

```

```

"SLURPLabelVocab":
    with open(path) as f:
        d = json.load(f)
        obj = cls.__new__(cls)
        obj.intent2id = d["intent2id"]
        obj.slot2id = d["slot2id"]
        obj.id2intent = {v: k for k, v in
obj.intent2id.items()}
        obj.id2slot = {v: k for k, v in
obj.slot2id.items()}
        return obj

```

```

#
=====
===== #
# 4. Audio I/O
#
=====
===== #
def load_and_resample(wav_path: str,
target_sr: int):
    import numpy as np
    waveform, sr =
torchaudio.load(wav_path)
    if waveform.shape[0] > 1:
        waveform = waveform.mean(dim=0,
keepdim=True)
    if sr != target_sr:
        waveform =
torchaudio.functional.resample(waveform,
sr, target_sr)
    return waveform.squeeze(0).numpy()

```

```

def _cache_key(rel_path: str) -> str:
    """Filesystem-safe cache filename for a
    wav_rel_path."""
    return
    hashlib.md5(rel_path.encode("utf-
8")).hexdigest() + ".pt"

#
=====
===== #
# 5. ASR transcript caching (parallel audio
decode + GPU batched decode)
#
=====
===== #
class ASRTranscriber:
    def __init__(self, model_name: str,
device: str, language: str = "en"):
        self.processor =
WhisperProcessor.from_pretrained(model_name
, local_files_only=True)
        self.model =
WhisperForConditionalGeneration.from_pretra
ined(model_name,
local_files_only=True).to(device)
        self.model.eval()

self.model.generation_config.forced_decoder
_ids = None
        self.device = device
        self.language = language

```

```

        @torch.no_grad()
        def transcribe_batch(self, waveforms:
List, sample_rate: int) -> List[str]:
            inputs = self.processor(
                waveforms,
                sampling_rate=sample_rate,
                return_tensors="pt",
                return_attention_mask=True
            )
            input_features =
inputs.input_features.to(self.device)
            attention_mask =
inputs.attention_mask.to(self.device)
            generated_ids =
self.model.generate(
                input_features,
                attention_mask=attention_mask,
                language=self.language,
                task="transcribe",
                num_beams=1, # PERF: greedy
decode -- some checkpoints default to beam
search in
                                #
generation_config, which multiplies ASR
decode time for no benefit
                                # here (we only
need a plausible noisy hypothesis, not the
best one).
            )
            return [t.strip() for t in
self.processor.batch_decode(generated_ids,
skip_special_tokens=True)]

        def build_cache(self, wav_paths:

```

```

List[str], root_dir: str, sample_rate: int,
cache_path: str,
        batch_size: int = 16,
num_workers: int = 4) -> Dict[str, str]:
    cache = {}
    if os.path.exists(cache_path):
        with open(cache_path) as f:
            cache = json.load(f)

    todo = [p for p in wav_paths if p
not in cache]
    print(f"[ASRTranscriber]
{len(todo)} / {len(wav_paths)} utterances
need decoding "
        f"(using {num_workers}
parallel decode workers)")
    if not todo:
        return cache

    # PERF: submit all audio-decode
jobs to a process pool up front. Because
futures
    # are submitted before we start
consuming them, the pool keeps decoding
batch i+1,
    # i+2, ... on other cores while the
GPU is busy running Whisper on batch i --
audio
    # I/O and GPU decode overlap
instead of serializing on one core.
    with
ProcessPoolExecutor(max_workers=num_workers
) as executor:
    full_paths =

```



```

[os.path.join(root_dir, p) for p in todo]
        futures =
[executor.submit(load_and_resample, fp,
sample_rate) for fp in full_paths]

        for i in range(0, len(todo),
batch_size):
            batch_paths = todo[i:i +
batch_size]
            batch_futures = futures[i:i
+ batch_size]
            waveforms = [f.result() for
f in batch_futures]
            hyps =
self.transcribe_batch(waveforms,
sample_rate)
            for p, h in
zip(batch_paths, hyps):
                cache[p] = h
                if (i // batch_size) % 20
== 0:
                    with open(cache_path,
"w") as f:
                        json.dump(cache, f)
                        print(f" ...{i +
len(batch_paths)}/{len(todo)} decoded")

                    with open(cache_path, "w") as f:
                        json.dump(cache, f)
                    return cache

#
=====

```

```

===== #
# 6. Acoustic feature caching (PERF: run
the frozen Whisper encoder ONCE
#   per recording, not once per training
step per epoch)
#
=====
===== #
def build_acoustic_feature_cache(
    encoder: "WhisperAcousticEncoder",
    feature_extractor:
WhisperFeatureExtractor,
    wav_paths: List[str],
    root_dir: str,
    sample_rate: int,
    cache_dir: str,
    device: str,
    batch_size: int = 32,
    num_workers: int = 4,
) -> None:
    os.makedirs(cache_dir, exist_ok=True)
    todo = [p for p in wav_paths if not
os.path.exists(os.path.join(cache_dir,
_cache_key(p)))]
    print(f"[acoustic-cache] {len(todo)} /
{len(wav_paths)} recordings need frozen-
encoder features")
    if not todo:
        return

    encoder.eval()
    with
ProcessPoolExecutor(max_workers=num_workers
) as executor:

```

```

        full_paths =
[os.path.join(root_dir, p) for p in todo]
        futures =
[executor.submit(load_and_resample, fp,
sample_rate) for fp in full_paths]

        for i in range(0, len(todo),
batch_size):
            batch_paths = todo[i:i +
batch_size]
            batch_futures = futures[i:i +
batch_size]
            waveforms = [f.result() for f
in batch_futures]

            inputs =
feature_extractor(waveforms,
sampling_rate=sample_rate,
return_tensors="pt")
            input_features =
inputs.input_features.to(device)

            with torch.no_grad():
                hidden =
encoder(input_features) # [B, T, H],
fixed T since Whisper pads to 30s
internally

                hidden = hidden.half().cpu()
                for j, p in
enumerate(batch_paths):
                    torch.save(hidden[j],
os.path.join(cache_dir, _cache_key(p)))

```

```

        if (i // batch_size) % 20 == 0:
            print(f" ...{i +
len(batch_paths)}/{len(todo)} features
cached")

#
=====
===== #
# 7. Slot-tag alignment caching (PERF:
# computed once, in parallel, instead
# of being recomputed inside collate_fn
# every batch of every epoch)
#
=====
===== #
def build_slot_tag_cache(
    examples: List[dict],          # each
    has_wav_rel_path, words, tags, asr_text
    cache_path: str,
    num_workers: int = 4,
) -> Dict[str, List[str]]:
    cache: Dict[str, List[str]] = {}
    if os.path.exists(cache_path):
        with open(cache_path) as f:
            cache = json.load(f)

    todo = [ex for ex in examples if
ex["wav_rel_path"] not in cache]
    print(f"[slot-tag-cache] {len(todo)} /
{len(examples)} recordings need tag
alignment")
    if todo:
        args = [(ex["words"], ex["tags"],

```

```

ex["asr_text"].split() or ["<empty>"]) for
ex in todo]
    with
ProcessPoolExecutor(max_workers=num_workers
) as executor:
    results =
list(executor.map(_align_worker, args,
chunksize=64))
    for ex, tags in zip(todo, results):
        cache[ex["wav_rel_path"]] =
tags
    with open(cache_path, "w") as f:
        json.dump(cache, f)
    return cache

#
=====
===== #
# 8. Dataset
#
=====
===== #
class SLURPDataset(Dataset):
    def __init__(
        self,
        jsonl_paths: List[Tuple[str, str]],
        root_dir: str,
        vocab: SLURPLabelVocab,
        sample_rate: int = 16000,
        max_audio_seconds: float = 11.0,
        asr_cache: Optional[Dict[str, str]]
= None,
        slot_tag_cache: Optional[Dict[str,

```

```

List[str]] = None,
        use_ground_truth_transcript: bool =
False,
        recording_filter: str =
"correct_only",
        recording_type_filter:
Optional[str] = None,
        use_precomputed_acoustic_features:
bool = True,
        acoustic_feature_cache_dir:
Optional[str] = None,
    ):
        self.root_dir = root_dir
        self.vocab = vocab
        self.sample_rate = sample_rate
        self.max_samples =
int(max_audio_seconds * sample_rate)
        self.asr_cache = asr_cache or {}
        self.slot_tag_cache =
slot_tag_cache or {}
        self.use_ground_truth_transcript =
use_ground_truth_transcript

        self.use_precomputed_acoustic_features =
use_precomputed_acoustic_features
        self.acoustic_feature_cache_dir =
acoustic_feature_cache_dir

        self.examples = []
        for jsonl_path, audio_subdir in
jsonl_paths:
            for entry in
_read_jsonl(jsonl_path):
                if entry["intent"] not in

```

```

vocab.intent2id:
    continue
    words, tags =
parse_sentence_annotation(entry.get("sentence_annotation", entry["sentence"]))

    for rec in
entry.get("recordings", []):
    if recording_filter ==
"correct_only" and rec.get("status") !=
"correct":
        continue
        fname = rec["file"]
        if
recording_type_filter == "headset" and "-
headset" not in fname:
            continue
            if
recording_type_filter == "non_headset" and
"-headset" in fname:
                continue

        self.examples.append({
            "wav_rel_path":
os.path.join(audio_subdir, fname),
            "words": words,
            "tags": tags,
            "sentence":
entry["sentence"],
            "intent_id":
vocab.intent2id[entry["intent"]],
        })

    def __len__(self):

```

```

        return len(self.examples)

    def __getitem__(self, idx: int) ->
Dict:
        ex = self.examples[idx]
        wav_rel_path = ex["wav_rel_path"]

        asr_text = ex["sentence"] if
self.use_ground_truth_transcript else \

self.asr_cache.get(wav_rel_path,
ex["sentence"])

        if
self.use_ground_truth_transcript:
            hyp_words, hyp_tags =
ex["words"], ex["tags"]
        else:
            hyp_words = asr_text.split() or
["<empty>"]
            # PERF: pull the precomputed
alignment instead of recomputing it here/in
collate_fn.
            hyp_tags =
self.slot_tag_cache.get(wav_rel_path)
            if hyp_tags is None:
                # fallback (e.g. cache not
built yet) -- computed lazily, still cheap
per-item
                hyp_tags =
_align_and_project_tags(ex["words"],
ex["tags"], hyp_words)

        item = {

```



```

        "wav_rel_path": wav_rel_path,
        "words": hyp_words,
        "tags": hyp_tags,
        "intent_id": ex["intent_id"],
    }

    if
self.use_precomputed_acoustic_features and
self.acoustic_feature_cache_dir is not
None:
        feat_path =
os.path.join(self.acoustic_feature_cache_dir, _cache_key(wav_rel_path))
        if os.path.exists(feat_path):
            # PERF: load a small cached
tensor instead of decoding audio + running
Whisper encoder.
            item["acoustic_features"] =
torch.load(feat_path).float()
            return item
            # fall through to raw audio if
this recording wasn't cached for any reason

        waveform =
load_and_resample(os.path.join(self.root_dir, wav_rel_path), self.sample_rate)
        if len(waveform) >
self.max_samples:
            start = random.randint(0,
len(waveform) - self.max_samples)
            waveform = waveform[start:start
+ self.max_samples]
            item["waveform"] = waveform
            return item

```

```

#
=====
===== #
# 9. Collator
#
=====
===== #
class SLURPCollator:
    def __init__(self, feature_extractor,
tokenizer, slot2id: Dict[str, int],
sample_rate: int = 16000,
max_text_len: int = 64):
        self.feature_extractor =
feature_extractor
        self.tokenizer = tokenizer
        self.slot2id = slot2id
        self.sample_rate = sample_rate
        self.max_text_len = max_text_len

    def __call__(self, batch: List[Dict]) -
> Dict[str, torch.Tensor]:
        out: Dict[str, torch.Tensor] = {}

        if "acoustic_features" in batch[0]:
            # PERF: all cached features
share the same fixed sequence length
(Whisper pads
            # every clip to 30s
internally), so this is a plain stack -- no
encoder forward,
            # no feature extraction, no
padding logic needed at train time.

```

```

        out["acoustic_states"] =
torch.stack([b["acoustic_features"] for b
in batch], dim=0)
    else:
        waveforms = [b["waveform"] for
b in batch]
        audio_inputs =
self.feature_extractor(waveforms,
sampling_rate=self.sample_rate,
return_tensors="pt")
        out["input_features"] =
audio_inputs.input_features

        all_words = [b["words"] for b in
batch]
        all_tags = [b["tags"] for b in
batch]

        text_inputs = self.tokenizer(
            all_words,
            is_split_into_words=True, padding=True,
            truncation=True,
            max_length=self.max_text_len,
            return_tensors="pt",
        )

        slot_labels =
torch.full(text_inputs["input_ids"].shape,
-100, dtype=torch.long)
        for i, tags in enumerate(all_tags):
            word_ids =
text_inputs.word_ids(batch_index=i)
            prev_word_idx = None
            for pos, w_idx in

```

```

enumerate(word_ids):
    if w_idx is None:
        continue
    if w_idx != prev_word_idx:
        tag = tags[w_idx] if
w_idx < len(tags) else "0"
        slot_labels[i, pos] =
self.slot2id.get(tag, self.slot2id["0"])
        prev_word_idx = w_idx

    out["input_ids"] =
text_inputs["input_ids"]
    out["text_attention_mask"] =
text_inputs["attention_mask"]
    out["slot_labels"] = slot_labels
    out["intent_labels"] =
torch.tensor([b["intent_id"] for b in
batch], dtype=torch.long)
    out["paths"] = [b["wav_rel_path"]
for b in batch]
    return out

#
=====
===== #
# 10. Model components
#
=====
===== #
class WhisperAcousticEncoder(nn.Module):
    def __init__(self, model_name: str =
"openai/whisper-small", layer: int = 6,
freeze: bool = True):

```

```

        super().__init__()
        self.whisper =
WhisperModel.from_pretrained(model_name,
local_files_only=True)
        self.layer = layer
        self.hidden_size =
self.whisper.config.d_model
        self._frozen = freeze
        if freeze:
            for p in
self.whisper.parameters():
                p.requires_grad = False
            self.whisper.eval()

    def forward(self, input_features:
torch.Tensor) -> torch.Tensor:
        ctx = torch.no_grad() if
self._frozen else torch.enable_grad()
        with ctx:
            enc_out =
self.whisper.encoder(input_features,
output_hidden_states=True,
return_dict=True)
            return
enc_out.hidden_states[self.layer]

class TextEncoder(nn.Module):
    def __init__(self, model_name: str =
"bert-base-uncased", freeze: bool = False):
        super().__init__()
        self.bert =
AutoModel.from_pretrained(model_name,
local_files_only=True)

```

```

        self.hidden_size =
self._get_hidden_size(self.bert.config)
        if freeze:
            for p in
self.bert.parameters():
                p.requires_grad = False

    @staticmethod
    def _get_hidden_size(config) -> int:
        for attr in ("hidden_size", "dim"):
            if hasattr(config, attr):
                return getattr(config,
attr)
            raise AttributeError(f"Could not
determine hidden size from config
{type(config).__name__}")

    def forward(self, input_ids,
attention_mask):
        return
self.bert(input_ids=input_ids,
attention_mask=attention_mask).last_hidden_
state

class CrossAttentionBlock(nn.Module):
    """
    Generic single-direction cross-
    attention block: `query_states` attends
    into
    `key_states`. Which modality plays
    which role is decided by the caller --
    this lets the same class implement both
    "text queries audio" (original

```

```
design) and "audio queries text" (the
swap) by just swapping which
tensor is passed as query vs. key.
"""
```

```
def __init__(self, query_dim, key_dim,
fusion_dim=768, num_heads=8, ffn_dim=2048,
dropout=0.1):
    super().__init__()
    self.query_proj =
nn.Linear(query_dim, fusion_dim)
    self.key_proj = nn.Linear(key_dim,
fusion_dim)
    self.cross_attn =
nn.MultiheadAttention(
        embed_dim=fusion_dim,
num_heads=num_heads, dropout=dropout,
batch_first=True
    )
    self.norm1 =
nn.LayerNorm(fusion_dim)
    self.ffn = nn.Sequential(
        nn.Linear(fusion_dim, ffn_dim),
nn.GELU(), nn.Dropout(dropout),
nn.Linear(ffn_dim, fusion_dim)
    )
    self.norm2 =
nn.LayerNorm(fusion_dim)
    self.dropout = nn.Dropout(dropout)

    def forward(self, query_states,
key_states, key_padding_mask=None):
        Q = self.query_proj(query_states)
        K = self.key_proj(key_states)
```

```

        attn_out, attn_weights =
self.cross_attn(
            query=Q, key=K, value=K,
key_padding_mask=key_padding_mask,
            need_weights=True,
average_attn_weights=True,
        )
        fused = self.norm1(Q +
self.dropout(attn_out))
        fused = self.norm2(fused +
self.dropout(self.ffn(fused)))
        return fused, attn_weights

def _masked_mean(x: torch.Tensor, mask:
torch.Tensor) -> torch.Tensor:
    m = mask.unsqueeze(-1).float()
    return (x * m).sum(1) /
m.sum(1).clamp(min=1e-6)

#
=====
===== #
# 11. Full model
#
=====
===== #
class SemiCascadedSLU_SLURP(nn.Module):
    """
    `fusion_mode` controls which modality
is the cross-attention query:

    "text_query" -- text queries audio

```


(original design). Fused output is
word-level: feeds
both slot_head directly and, pooled
over text
positions, intent_head.

"audio_query" -- audio queries text.
Fused output is frame-level: feeds
intent_head
directly (mean-pooled over frames, which is
valid since Whisper
always produces a fixed-length
frame sequence). A
second block then has TEXT query
into that frame-
level representation to recover a
word-level
representation for slot_head, since the
slot task
inherently needs one prediction per word
and a frame-indexed
sequence has no such alignment.

"dual_branch" -- both blocks run
independently: the text-query branch
feeds slot_head
(identical to "text_query"), the
audio-query branch
feeds intent_head (identical
pooling to
"audio_query"). No secondary projection is
needed here because
each branch only serves the head
that matches its

```

native granularity.
"""

    def __init__(
        self,
        num_intents: int,
        num_slot_labels: int,
        whisper_model_name: str =
"openai/whisper-small",
        bert_model_name: str = "bert-base-
uncased",
        whisper_layer: int = 6,
        fusion_dim: int = 768,
        freeze_whisper: bool = True,
        freeze_bert: bool = False,
        dropout: float = 0.1,
        fusion_mode: str = "text_query",
    ):
        super().__init__()
        assert fusion_mode in
("text_query", "audio_query",
"dual_branch"), fusion_mode
        self.fusion_mode = fusion_mode

        self.acoustic_encoder =
WhisperAcousticEncoder(whisper_model_name,
whisper_layer, freeze_whisper)
        self.text_encoder =
TextEncoder(bert_model_name, freeze_bert)
        text_dim =
self.text_encoder.hidden_size
        acoustic_dim =
self.acoustic_encoder.hidden_size

```

```

        if fusion_mode in ("text_query",
                           "dual_branch"):
            self.text_query_block =
CrossAttentionBlock(text_dim, acoustic_dim,
fusion_dim, dropout=dropout)
            self.text_gate =
nn.Sequential(nn.Linear(fusion_dim * 2,
fusion_dim), nn.Sigmoid())

        if fusion_mode in ("audio_query",
                           "dual_branch"):
            self.audio_query_block =
CrossAttentionBlock(acoustic_dim, text_dim,
fusion_dim, dropout=dropout)
            self.audio_gate =
nn.Sequential(nn.Linear(fusion_dim * 2,
fusion_dim), nn.Sigmoid())

        if fusion_mode == "audio_query":
            # secondary block: text queries
into the (fusion_dim-sized) frame-level
            # audio-query output, to
recover a word-level representation for
slot_head.
            self.slot_projection_block =
CrossAttentionBlock(text_dim, fusion_dim,
fusion_dim, dropout=dropout)
            self.slot_gate2 =
nn.Sequential(nn.Linear(fusion_dim * 2,
fusion_dim), nn.Sigmoid())

        self.dropout = nn.Dropout(dropout)
        self.intent_head =
nn.Linear(fusion_dim, num_intents)

```

```

        self.slot_head =
nn.Linear(fusion_dim, num_slot_labels)

    def forward(
        self,
        input_features:
Optional[torch.Tensor] = None,
        acoustic_states:
Optional[torch.Tensor] = None,    # PERF:
precomputed frozen-encoder output
        input_ids: torch.Tensor = None,
        text_attention_mask: torch.Tensor =
None,
        acoustic_attention_mask:
torch.Tensor = None,
        intent_labels: torch.Tensor = None,
        slot_labels: torch.Tensor = None,
        slot_loss_weight: float = 1.0,
        **kwargs,
    ):
        if acoustic_states is None:
            assert input_features is not
None, "must supply either input_features or
precomputed acoustic_states"
            acoustic_states =
self.acoustic_encoder(input_features)
            text_states =
self.text_encoder(input_ids,
text_attention_mask)

            text_key_padding_mask =
~text_attention_mask.bool()
            acoustic_key_padding_mask =
~acoustic_attention_mask.bool() if

```

```

acoustic_attention_mask is not None else
None

        intent_logits = None
        slot_logits = None
        extra: Dict[str, torch.Tensor] = {}

        if self.fusion_mode in
("text_query", "dual_branch"):
            fused_t, attn_t =
self.text_query_block(text_states,
acoustic_states,
key_padding_mask=acoustic_key_padding_mask)
            text_proj =
self.text_query_block.query_proj(text_state
s)

            g_t =
self.text_gate(torch.cat([fused_t,
text_proj], dim=-1))
            mixed_t = self.dropout(g_t *
fused_t + (1 - g_t) * text_proj)
            slot_logits =
self.slot_head(mixed_t)
            extra["text_query_attn"] =
attn_t

            extra["text_gate"] = g_t
            if self.fusion_mode ==
"text_query":
                pooled =
_masked_mean(mixed_t, text_attention_mask)
                intent_logits =
self.intent_head(pooled)

            if self.fusion_mode in

```

```

("audio_query", "dual_branch"):
    frame_fused, attn_a =
self.audio_query_block(acoustic_states,
text_states,
key_padding_mask=text_key_padding_mask)
    acoustic_proj =
self.audio_query_block.query_proj(acoustic_
states)

    g_a =
self.audio_gate(torch.cat([frame_fused,
acoustic_proj], dim=-1))
    mixed_a = self.dropout(g_a *
frame_fused + (1 - g_a) * acoustic_proj)
    pooled_a =
mixed_a.mean(dim=1) # fixed-length frame
sequence, no padding to mask out
    intent_logits =
self.intent_head(pooled_a)
    extra["audio_query_attn"] =
attn_a

    extra["audio_gate"] = g_a

    if self.fusion_mode ==
"audio_query":
        word_fused, attn_t2 =
self.slot_projection_block(text_states,
mixed_a)
        text_proj2 =
self.slot_projection_block.query_proj(text_
states)
        g_t2 =
self.slot_gate2(torch.cat([word_fused,
text_proj2], dim=-1))
        mixed_t2 =

```

```

self.dropout(g_t2 * word_fused + (1 - g_t2)
* text_proj2)
        slot_logits =
self.slot_head(mixed_t2)

extra["slot_projection_attn"] = attn_t2

        output = {
            "intent_logits": intent_logits,
            "slot_logits": slot_logits,
            **extra,
        }

        if intent_labels is not None and
slot_labels is not None:
            intent_loss =
F.cross_entropy(intent_logits,
intent_labels)
            slot_loss = F.cross_entropy(
                slot_logits.view(-1,
slot_logits.size(-1)),
slot_labels.view(-1), ignore_index=-100
            )
            output["loss"] = intent_loss +
slot_loss_weight * slot_loss
            output["intent_loss"] =
intent_loss
            output["slot_loss"] = slot_loss

        return output

#
=====

```

```

===== #
# 12. Metrics (unchanged)
#
=====
===== #
def _intent_metrics(y_true: List[int],
y_pred: List[int]) -> Dict[str, float]:
    from sklearn.metrics import
precision_recall_fscore_support,
accuracy_score
    acc = accuracy_score(y_true, y_pred)
    p_macro, r_macro, f1_macro, _ =
precision_recall_fscore_support(y_true,
y_pred, average="macro", zero_division=0)
    p_w, r_w, f1_w, _ =
precision_recall_fscore_support(y_true,
y_pred, average="weighted",
zero_division=0)
    return {
        "intent_acc": acc,
        "intent_precision_macro": p_macro,
        "intent_recall_macro": r_macro,
        "intent_f1_macro": f1_macro,
        "intent_precision_weighted": p_w,
        "intent_recall_weighted": r_w,
        "intent_f1_weighted": f1_w,
    }

def _bio_to_spans(tags: List[str]) ->
List[Tuple[str, int, int]]:
    spans = []
    start, ent_type = None, None
    for i, tag in enumerate(tags + ["0"]):

```



```

        if tag.startswith("B-"):
            if start is not None:
                spans.append((ent_type,
start, i))
                start, ent_type = i, tag[2:]
            elif tag.startswith("I-") and
ent_type == tag[2:]:
                continue
            else:
                if start is not None:
                    spans.append((ent_type,
start, i))
                    start, ent_type = None,
None
                if tag.startswith("I-"):
                    start, ent_type = i,
tag[2:]
            return spans

def _slot_span_metrics(all_true_tags:
List[List[str]], all_pred_tags:
List[List[str]]) -> Dict[str, float]:
    tp = fp = fn = 0
    for true_tags, pred_tags in
zip(all_true_tags, all_pred_tags):
        true_spans =
set(_bio_to_spans(true_tags))
        pred_spans =
set(_bio_to_spans(pred_tags))
        tp += len(true_spans & pred_spans)
        fp += len(pred_spans - true_spans)
        fn += len(true_spans - pred_spans)
    precision = tp / (tp + fp) if (tp + fp)

```

```

> 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) >
0 else 0.0
    f1 = 2 * precision * recall /
(precision + recall) if (precision +
recall) > 0 else 0.0
    return {"slot_precision": precision,
"slot_recall": recall, "slot_f1": f1}

def _forward_batch(model: nn.Module, batch:
Dict) -> Dict:
    kwargs = dict(
        input_ids=batch["input_ids"],

text_attention_mask=batch["text_attention_m
ask"],

intent_labels=batch["intent_labels"],
        slot_labels=batch["slot_labels"],
    )
    if "acoustic_states" in batch:
        kwargs["acoustic_states"] =
batch["acoustic_states"]
    else:
        kwargs["input_features"] =
batch["input_features"]
    return model(**kwargs)

@torch.no_grad()
def evaluate(model: nn.Module, loader:
DataLoader, device: str, vocab:
SLURPLabelVocab) -> Dict[str, float]:

```

```

model.eval()
total, correct_intent = 0, 0
total_loss = 0.0
all_intent_true, all_intent_pred = [],
[]
all_slot_true_tags, all_slot_pred_tags
= [], []

for batch in loader:
    batch = {k: (v.to(device) if
torch.is_tensor(v) else v) for k, v in
batch.items()}
    out = _forward_batch(model, batch)

    bsz =
batch["intent_labels"].size(0)
    total_loss += out["loss"].item() *
bsz
    total += bsz

    pred_intent =
out["intent_logits"].argmax(-1)
    correct_intent += (pred_intent ==
batch["intent_labels"]).sum().item()
    all_intent_true +=
batch["intent_labels"].cpu().tolist()
    all_intent_pred +=
pred_intent.cpu().tolist()

    pred_slot_ids =
out["slot_logits"].argmax(-1)
    slot_labels = batch["slot_labels"]
    mask = slot_labels != -100
    for i in range(bsz):

```

```

        true_ids = slot_labels[i]
    [mask[i]].cpu().tolist()
        pred_ids = pred_slot_ids[i]
    [mask[i]].cpu().tolist()

    all_slot_true_tags.append([vocab.id2slot[t]
    for t in true_ids])

    all_slot_pred_tags.append([vocab.id2slot[p]
    for p in pred_ids])

    metrics = {"loss": total_loss / total,
    "intent_acc": correct_intent / total}

    metrics.update(_intent_metrics(all_intent_t
    rue, all_intent_pred))

    metrics.update(_slot_span_metrics(all_slot_
    true_tags, all_slot_pred_tags))
    return metrics

#
=====
===== #
# 13. Data pipeline
#
=====
===== #
def build_data loaders(cfg: SLURPConfig) ->
    Tuple[DataLoader, DataLoader, DataLoader,
    SLURPLabelVocab]:
    train_jsonl =
    os.path.join(cfg.root_dir, cfg.train_jsonl)

```

```

        valid_jsonl =
os.path.join(cfg.root_dir, cfg.valid_jsonl)
        test_jsonl = os.path.join(cfg.root_dir,
cfg.test_jsonl)
        synth_jsonl =
os.path.join(cfg.root_dir,
cfg.train_synthetic_jsonl)

        vocab_source_paths = [train_jsonl,
valid_jsonl, test_jsonl]
        if cfg.include_synthetic and
os.path.exists(synth_jsonl):

vocab_source_paths.append(synth_jsonl)
        vocab =
SLURPLabelVocab(vocab_source_paths)
        vocab.save("slurp_label_vocab.json")
        print(f"[vocab] intents=
{vocab.num_intents} slot_labels=
{vocab.num_slot_labels}")

        # --- gather (wav_rel_path, words,
tags, sentence) for every recording up
front ---
        all_jsonl_specs = [(train_jsonl,
cfg.audio_real_subdir),
                            (valid_jsonl,
cfg.audio_real_subdir),
                            (test_jsonl,
cfg.audio_real_subdir)]
        if cfg.include_synthetic and
os.path.exists(synth_jsonl):

all_jsonl_specs.append((synth_jsonl,

```

```

cfg.audio_synth_subdir))

    all_recordings = [] # list of dicts:
    wav_rel_path, words, tags, sentence
    for jp, subdir in all_jsonl_specs:
        for entry in _read_jsonl(jp):
            if entry["intent"] not in
vocab.intent2id:
                continue
            words, tags =
parse_sentence_annotation(entry.get("senten
ce_annotation", entry["sentence"]))
            for rec in
entry.get("recordings", []):
                if cfg.recording_filter ==
"correct_only" and rec.get("status") !=
"correct":
                    continue
                all_recordings.append({
                    "wav_rel_path":
os.path.join(subdir, rec["file"]),
                    "words": words,
                    "tags": tags,
                    "sentence":
entry["sentence"],
                })

    all_paths = [r["wav_rel_path"] for r in
all_recordings]

    # --- 1) ASR transcript cache (parallel
decode) ---
    asr_cache = {}
    if not cfg.use_ground_truth_transcript:

```

```

        transcriber =
ASRTranscriber(cfg.whisper_model_name,
cfg.device)
        asr_cache =
transcriber.build_cache(
            all_paths, cfg.root_dir,
cfg.sample_rate, cfg.asr_cache_path,
            batch_size=cfg.prep_batch_size,
num_workers=cfg.prep_num_workers,
        )
        del transcriber
        if cfg.device == "cuda":
            torch.cuda.empty_cache()

        feature_extractor =
WhisperFeatureExtractor.from_pretrained(cfg
.whisper_model_name, local_files_only=True)
        tokenizer =
AutoTokenizer.from_pretrained(cfg.bert_model_name, local_files_only=True)

        # --- 2) frozen acoustic feature cache
        ---
        if
cfg.use_precomputed_acoustic_features and
cfg.freeze_whisper:
            cache_encoder =
WhisperAcousticEncoder(cfg.whisper_model_name,
cfg.whisper_layer,
freeze=True).to(cfg.device)
            build_acoustic_feature_cache(
                cache_encoder,
feature_extractor, all_paths, cfg.root_dir,
cfg.sample_rate,

```

```

        cfg.acoustic_feature_cache_dir,
        cfg.device,
        batch_size=cfg.prep_batch_size,
        num_workers=cfg.prep_num_workers,
    )
    del cache_encoder
    if cfg.device == "cuda":
        torch.cuda.empty_cache()
    elif
cfg.use_precomputed_acoustic_features and
not cfg.freeze_whisper:
        print("[warn]
use_precomputed_acoustic_features=True but
freeze_whisper=False -- "
            "acoustic encoder is
trainable, so its output can't be cached.
Disabling feature cache.")

cfg.use_precomputed_acoustic_features =
False

    # --- 3) slot-tag alignment cache ---
    slot_tag_cache = {}
    if not cfg.use_ground_truth_transcript:
        for r in all_recordings:
            r["asr_text"] =
asr_cache.get(r["wav_rel_path"],
r["sentence"])
            slot_tag_cache =
build_slot_tag_cache(all_recordings,
cfg.slot_tag_cache_path,
cfg.prep_num_workers)

    collator = SLURPCollator(

```



```

        feature_extractor, tokenizer,
vocab.slot2id,
        sample_rate=cfg.sample_rate,
max_text_len=cfg.max_text_len,
    )

    common_kwargs = dict(
        root_dir=cfg.root_dir, vocab=vocab,
sample_rate=cfg.sample_rate,

max_audio_seconds=cfg.max_audio_seconds,
asr_cache=asr_cache,
slot_tag_cache=slot_tag_cache,

use_ground_truth_transcript=cfg.use_ground_
truth_transcript,

recording_filter=cfg.recording_filter,
recording_type_filter=cfg.recording_type_fi
lter,

use_precomputed_acoustic_features=cfg.use_p
recomputed_acoustic_features,

acoustic_feature_cache_dir=cfg.acoustic_fea
ture_cache_dir,
    )

    train_jsonl_paths = [(train_jsonl,
cfg.audio_real_subdir)]
    if cfg.include_synthetic and
os.path.exists(synth_jsonl):

train_jsonl_paths.append((synth_jsonl,

```

```

cfg.audio_synth_subdir))

    train_ds =
SLURPDataset(train_jsonl_paths,
**common_kwargs)
    valid_ds = SLURPDataset([(valid_jsonl,
cfg.audio_real_subdir)], **common_kwargs)
    test_ds = SLURPDataset([(test_jsonl,
cfg.audio_real_subdir)], **common_kwargs)
    print(f"[data] train={len(train_ds)}
valid={len(valid_ds)} test={len(test_ds)}
examples (per-recording)")

    # PERF: persistent_workers avoids re-
spawning worker processes every epoch;
    # pin_memory speeds up host->GPU
transfer. Worker cost is now trivial
    # (load one small cached tensor +
tokenize text) since audio decode /
    # Whisper-encoder forward / tag
alignment all happened once above.
    dl_kwargs =
dict(num_workers=cfg.num_workers,
collate_fn=collator, pin_memory=(cfg.device
== "cuda"))
    if cfg.num_workers > 0:
        dl_kwargs["persistent_workers"] =
True

    train_loader = DataLoader(train_ds,
batch_size=cfg.batch_size, shuffle=True,
drop_last=True, **dl_kwargs)
    valid_loader = DataLoader(valid_ds,
batch_size=cfg.batch_size, shuffle=False,

```

```

**dl_kwargs)
    test_loader = DataLoader(test_ds,
batch_size=cfg.batch_size, shuffle=False,
**dl_kwargs)
    return train_loader, valid_loader,
test_loader, vocab

#
=====
===== #
# 14. Training loop
#
=====
===== #
def build_model(cfg: SLURPConfig, vocab:
SLURPLabelVocab, fusion_mode: Optional[str]
= None) -> nn.Module:
    mode = fusion_mode or cfg.fusion_mode
    return SemiCascadedSLU_SLURP(
        num_intents=vocab.num_intents,

num_slot_labels=vocab.num_slot_labels,

whisper_model_name=cfg.whisper_model_name,

bert_model_name=cfg.bert_model_name,
        whisper_layer=cfg.whisper_layer,
        freeze_whisper=cfg.freeze_whisper,
        freeze_bert=False,
        fusion_mode=mode,
    ).to(cfg.device)

```

```

def train_model(
    model: nn.Module,
    train_loader: DataLoader,
    valid_loader: DataLoader,
    test_loader: DataLoader,
    vocab: SLURPLabelVocab,
    cfg: SLURPConfig,
    run_tag: str = "",
) -> Dict[str, float]:
    """Runs the epoch loop for an already-
    built model and returns final test metrics.
    Split out from `train()` so the same
    cached dataloaders can be reused to train
    several fusion_mode variants back-to-
    back without rebuilding ASR/feature/tag
    caches each time -- see
    `run_comparison`. """
    ckpt_name =
    f"slurp_semi_cascaded_slu_best_{run_tag}.pt"
    " if run_tag else
    "slurp_semi_cascaded_slu_best.pt"

    trainable = [p for p in
    model.parameters() if p.requires_grad]
    optimizer =
    torch.optim.AdamW(trainable, lr=cfg.lr)

    best_valid_f1 = -1.0
    for epoch in range(cfg.epochs):
        model.train()
        running_loss = 0.0
        for step, batch in
        enumerate(train_loader):
            batch = {k: (v.to(cfg.device,

```

```

non_blocking=True) if torch.is_tensor(v)
else v) for k, v in batch.items()
        out = _forward_batch(model,
batch)

        out_loss = out["loss"]
        optimizer.zero_grad()
        out_loss.backward()

torch.nn.utils.clip_grad_norm_(trainable,
max_norm=1.0)
        optimizer.step()

        running_loss += out_loss.item()
        if step % 50 == 0:
            tag = f"[{run_tag}] " if
run_tag else ""
            print(f"{tag}epoch {epoch}
step {step} loss {out_loss.item():.4f} "
                  f"(intent=
{out['intent_loss'].item():.4f} slot=
{out['slot_loss'].item():.4f})")

            valid_metrics = evaluate(model,
valid_loader, cfg.device, vocab)
            tag = f"[{run_tag}] " if run_tag
else ""
            print(f"{tag}[epoch {epoch}]
train_loss={running_loss /
len(train_loader):.4f} valid=
{valid_metrics}")

            combined = 0.5 *
valid_metrics["intent_acc"] + 0.5 *
valid_metrics["slot_f1"]

```

```

        if combined > best_valid_f1:
            best_valid_f1 = combined
            torch.save(model.state_dict(),
ckpt_name)
            print(f"{tag} -> new best
(intent_acc+slot_f1 avg=
{best_valid_f1:.4f}), checkpoint saved to
{ckpt_name}")

        test_metrics = evaluate(model,
test_loader, cfg.device, vocab)
        print(f"{tag}[final test]
{test_metrics}")
        return test_metrics

def train(cfg: SLURPConfig):
    """Single-architecture training entry
point (uses cfg.fusion_mode)."""
    train_loader, valid_loader,
test_loader, vocab = build_dataloaders(cfg)
    model = build_model(cfg, vocab)
    test_metrics = train_model(model,
train_loader, valid_loader, test_loader,
vocab, cfg, run_tag=cfg.fusion_mode)
    return model, vocab, test_metrics

def run_comparison(cfg: SLURPConfig,
fusion_modes: Optional[List[str]] = None) -
> Dict[str, Dict[str, float]]:
    """Trains each fusion_mode variant on
identical data/splits (dataloaders and all
caches -- ASR transcripts, frozen

```

```

acoustic features, slot-tag alignments --
are
    built ONCE and reused across variants)
and prints a comparison table at the
end."""
    modes = list(fusion_modes or
cfg.comparison_modes)
    train_loader, valid_loader,
test_loader, vocab = build_dataloaders(cfg)

    results: Dict[str, Dict[str, float]] =
{}
    for mode in modes:
        print(f"\n{'=' * 70}\n=== training
fusion_mode={mode} ===\n{'=' * 70}")
        model = build_model(cfg, vocab,
fusion_mode=mode)
        n_params = sum(p.numel() for p in
model.parameters() if p.requires_grad)
        print(f"[{mode}] trainable params:
{n_params:,}")
        results[mode] = train_model(model,
train_loader, valid_loader, test_loader,
vocab, cfg, run_tag=mode)
        del model
        if cfg.device == "cuda":
            torch.cuda.empty_cache()

    print(f"\n{'=' * 70}\n=== comparison:
{'', '.join(modes)} ===\n{'=' * 70}")
    header = f"{'fusion_mode':<14}
{'intent_acc':>12}{ 'intent_f1_macro':>18}
{'slot_f1':>10}{ 'slot_precision':>16}
{'slot_recall':>13}"

```

```

print(header)
print("-" * len(header))
for mode in modes:
    m = results[mode]
    print(f"{mode:<14}"
          {m['intent_acc']:>12.4f}
          {m['intent_f1_macro']:>18.4f}"
          f"{m['slot_f1']:>10.4f}
          {m['slot_precision']:>16.4f}
          {m['slot_recall']:>13.4f}")

    best_mode = max(results, key=lambda k:
0.5 * results[k]["intent_acc"] + 0.5 *
results[k]["slot_f1"])
    print(f"\nBest by 0.5*intent_acc +
0.5*slot_f1: {best_mode}")
    return results

if __name__ == "__main__":
    cfg = SLURPConfig(
        root_dir="slurp_dataset",
        use_ground_truth_transcript=False,
        recording_filter="correct_only",

        use_precomputed_acoustic_features=True,
        batch_size=8,
        epochs=10,
    )
    # Trains "text_query" (baseline),
    "audio_query" (full swap), and
    "dual_branch"
    # back-to-back on identical cached data
    and prints a side-by-side comparison.

```



```
# To train just one architecture  
instead, set cfg.fusion_mode and call  
train(cfg).  
run_comparison(cfg)
```